



BSM 304

İşletim Sistemleri

5. Hafta – Thread (İş Parçacığı)

Dr. Öğr. Üyesi Onur ÇAKIRGÖZ
onurcakirgoz@bartin.edu.tr

BÖLÜM - 5

Bu bölümde,

- Thread (İş Parçacığı)
- Thread'lerin Faydaları
- Multi-core Programlama
- Amdahl Yasası
- Paralel Çalışma Türleri
- Thread Kütüphaneleri
- Pthreads

konularına değinilecektir.

Thread (İş Parçacığı)

- Bir thread,
 - Thread Id
 - Program sayacı,
 - Bir grup register ve
 - Bir stack yapısını içerir.
- Thread'ler, program kodunu, data kısmını, dosyalar gibi işletim sistemi kaynaklarını **ortak kullanır**.

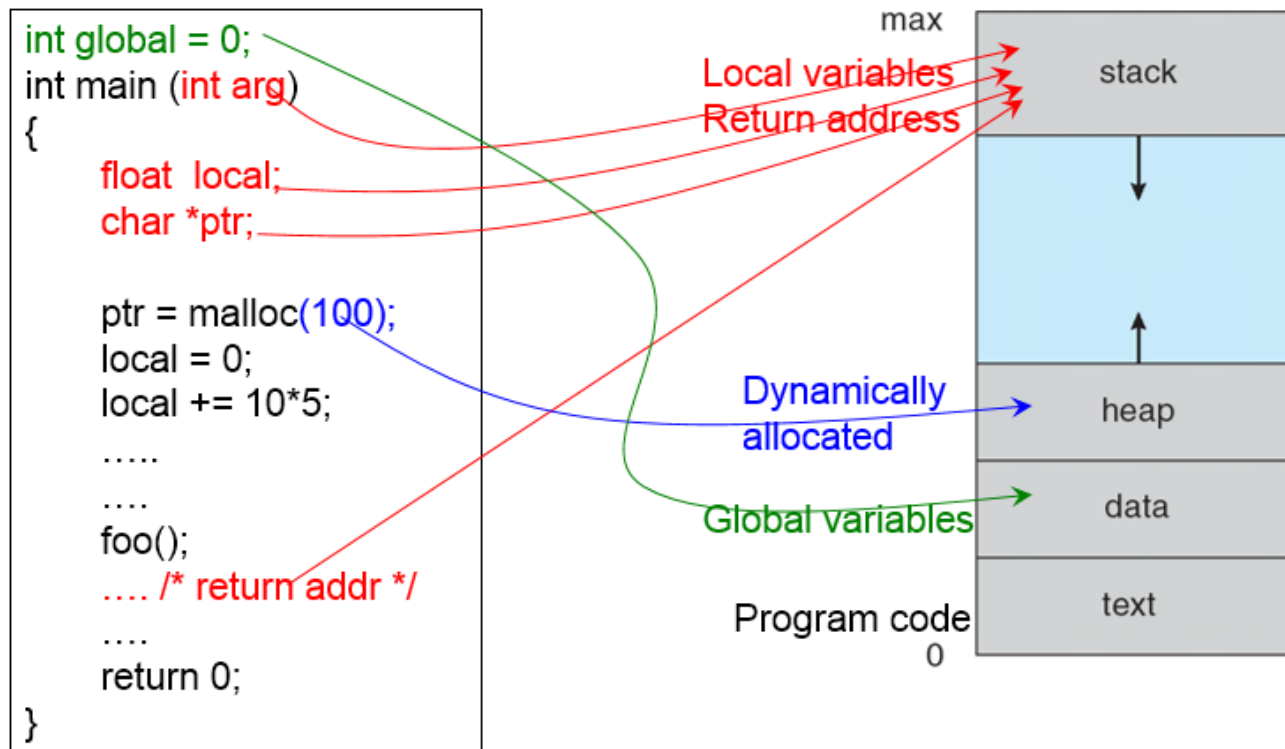
Thread (İş Parçacığı) (devam...)

- Klasik prosesler **tek thread'e** sahiptirler.
- Eğer bir proses, **birden fazla thread'e sahipse** **birden fazla görevi eşzamanlı yapabilir.**
- Günümüzdeki modern bilgisayarlarda çalışan yazılım uygulamalarının **çoğu multithread** çalışırlar.

Thread (İş Parçacığı) (devam...)

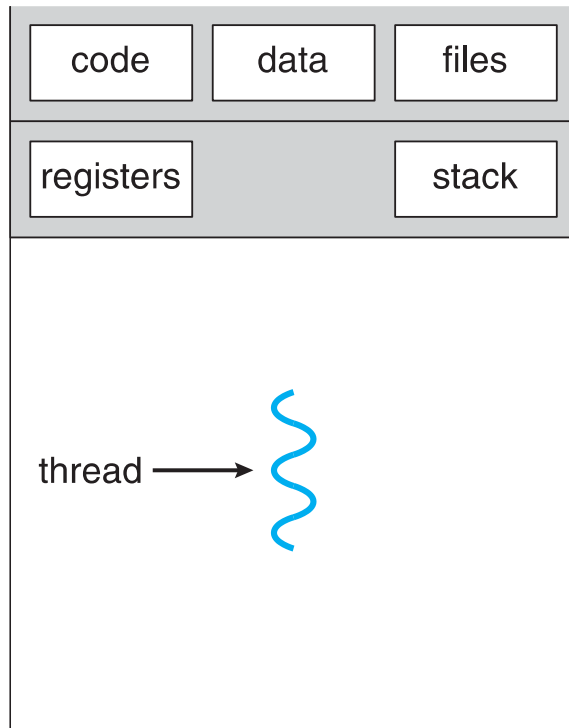
- Günümüzdeki uygulamalar, çok sayıda thread'e sahip **tek proses** şeklinde geliştirilirler.
- Bir **web tarayıcı**,
 - bir thread ile **veri aktarımı** yapabilir,
 - başka thread ile **verileri ekranda görüntüleyebilir**.
- Bir **kelime işleme (Microsoft Word)** uygulaması,
 - bir thread ile **klavyeden giriş alabilir**,
 - bir thread ile **spell check yapabilir** ve
 - başka bir thread ile **ekran görüntüsünü** düzenleyebilir.
- Çoğu işletim sistemi kernel'ı multi-threaded yapıdadır.

Proses Üzerindeki Bilgiler (HATIRLAYALIM)

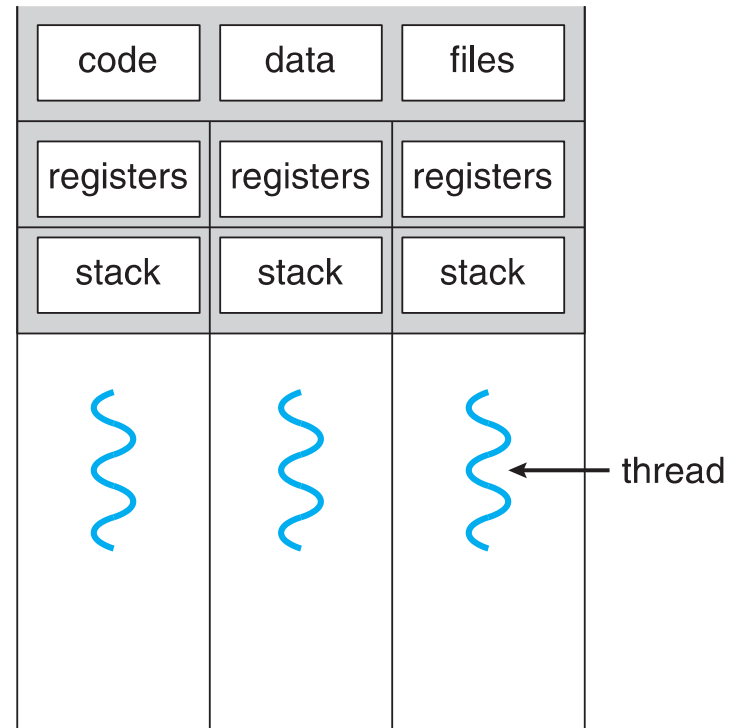


Thread (İş Parçacığı) (devam...)

- Her thread, **paylaşmadan kullandığı kendisine ait bileşenlere** sahiptir. **Single ve multi-threaded proses:**



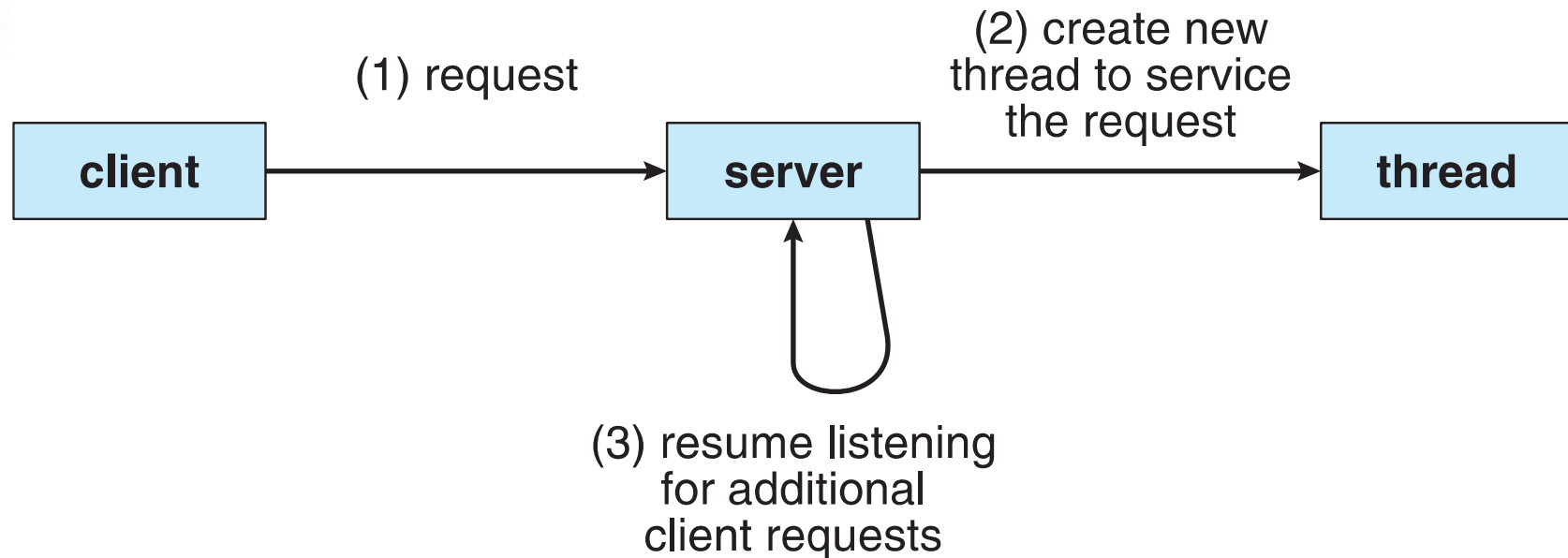
single-threaded process



multithreaded process

Multi-thread Sunucu Mimarisi

- Bir Web sunucu prosesi **multithreaded çalışırsa**, her gelen istek için **ayrı bir thread oluşturulur** ve proses portu dinlemeye devam eder.
- Her istek için ayrı bir proses oluşturmak **hem zaman alır** hem de **maliyetlidir**.



Thread'lerin Faydaları

- Thread kullanımının faydalarını 4 ana başlıkta toplayabiliriz:

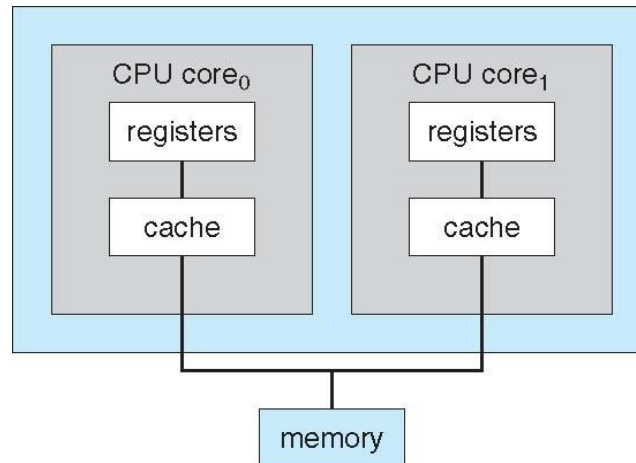
1. **Responsiveness (cevap verebilirlik)**: Kullanıcı etkileşimli uygulamalarda, bir kısım bloklanmış, kilitlenmiş veya uzun süren işlem **yürütülse bile** kullanıcı ile etkileşim yapan kısmı çalışmasını sürdürür. **Sistemin cevap verebilirlik özelliği artmış olur. (Sunucu mimarisi)**
2. **Resource sharing (kaynak paylaşımı)**: Prosesler kaynaklarını shared memory veya message passing teknikleri aracılığıyla paylaşabilirler. **Thread'ler ait oldukları prosesin sahip olduğu hafıza alanını ve diğer kaynakları paylaşabilirler.**

Thread'lerin Faydaları (devam...)

3. **Economy**: Bir **proses** oluştururken hafıza ve kaynak tahsis edilmesi maliyeti yüksek bir iştir. **Thread'ler** ait oldukları prosesin kaynaklarını paylaştıklarından dolayı **context switch daha düşük maliyetle** yapılır. (Solaris işletim sisteminde, thread oluşturma 30 kat daha hızlıdır ve **thread'lerde context switch** 5 kat daha hızlıdır.)
4. **Scalibility (Ölçeklenebilirlik)**: Çok işlemcili mimarilerde thread'ler **farklı core'lar üzerinde eşzamanlı çalışabilir**. Ancak, tek thread yapısına sahip process sadece bir işlemci üzerinde çalışabilir. (Sistemin kullanıcı sayısı arttıkça (100x veya daha fazla kat), yazılım makul yanıt süreleriyle çalışmaya devam edecektir.)

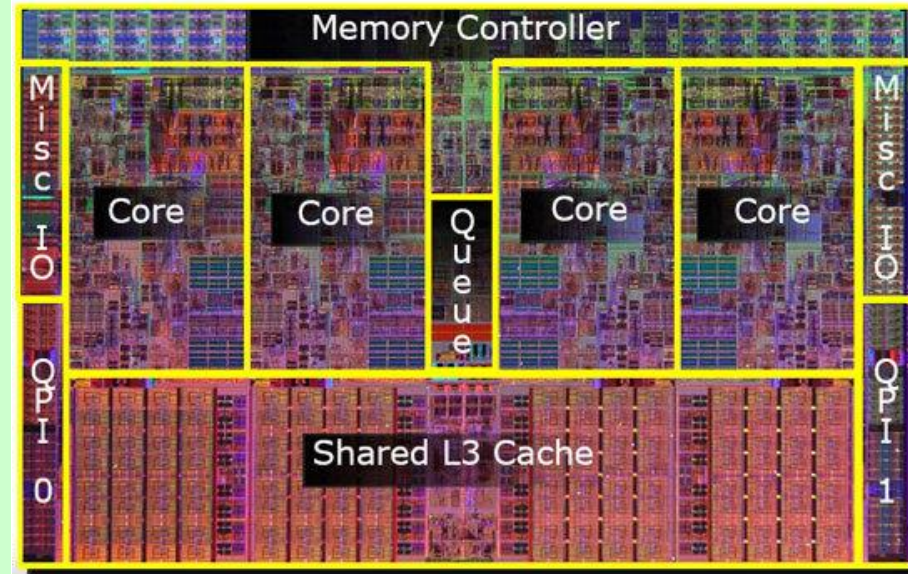
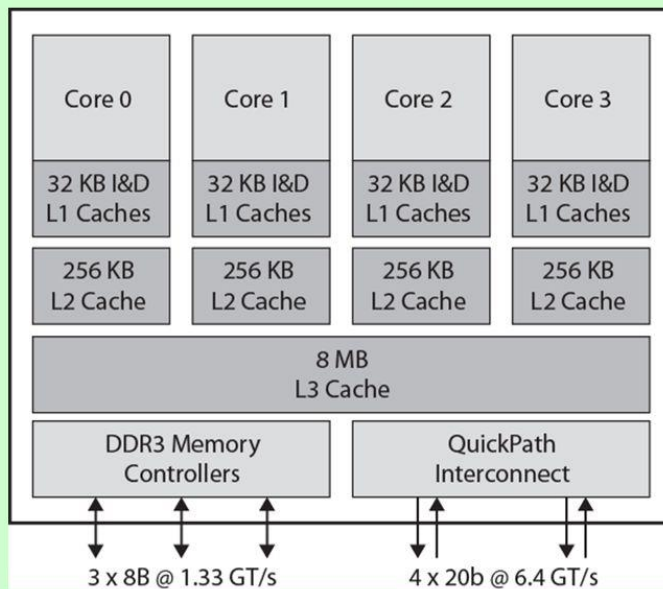
Multi-core (Çok çekirdekli) Programlama

- Bilgisayar mimarisindeki en önemli gelişmelerden birisi, **çok işlemcili sistemlerin geliştirilmesidir**.
- Son zamanlarda, **tek chip içerisine birden fazla core yerleştirilmektedir** ve bu tür sistemler **multi-core** olarak adlandırılır.
- Her bir core **işletim sistemi için ayrı bir işlemci** olarak görünür.



Günümüz İşlemci Yapısı

Intel Core i7 Block Diagram



L3 Önbelleğin Çekirdekler Tarafından Paylaşılmasının Sebepleri - Hatırlayalım

- L3 önbelleğin çekirdekler tarafından paylaşılmasının asıl sebeplerinden biri, modern işlemcilerin thread'ler (iş parçacıkları) ve paralel programlama için optimize edilmesidir. Ancak bu durumun altında yatan birkaç teknik neden ve avantaj daha vardır:
- Paralel programlamada çekirdeklerin bazı ortak verilere erişme ihtimali yüksektir. L3 önbellek paylaşımı, bu ortak verilerin RAM'e gidilmeden hızlıca erişilebilir olmasını sağlar.
- L3 önbellek, çekirdekler arasındaki veri paylaşımı için merkezi bir nokta sağlar ve tutarlılık protokollerini daha verimli kılar.
- Çok çekirdekli işlemcilerde, thread migration işlemi çok daha verimli olur.
- Bazı çekirdekler daha fazla önbellek gerektirirken diğerleri daha az kullanabilir. Bu esneklik, sistemin genel verimliliğini artırır.
- Bir çekirdek bir thread'in çalışmasını tamamladıktan sonra, başka bir çekirdek L3 önbellekteki aynı veriyi veya komutları kullanabilir.

Multi-core Programlama (devam...)

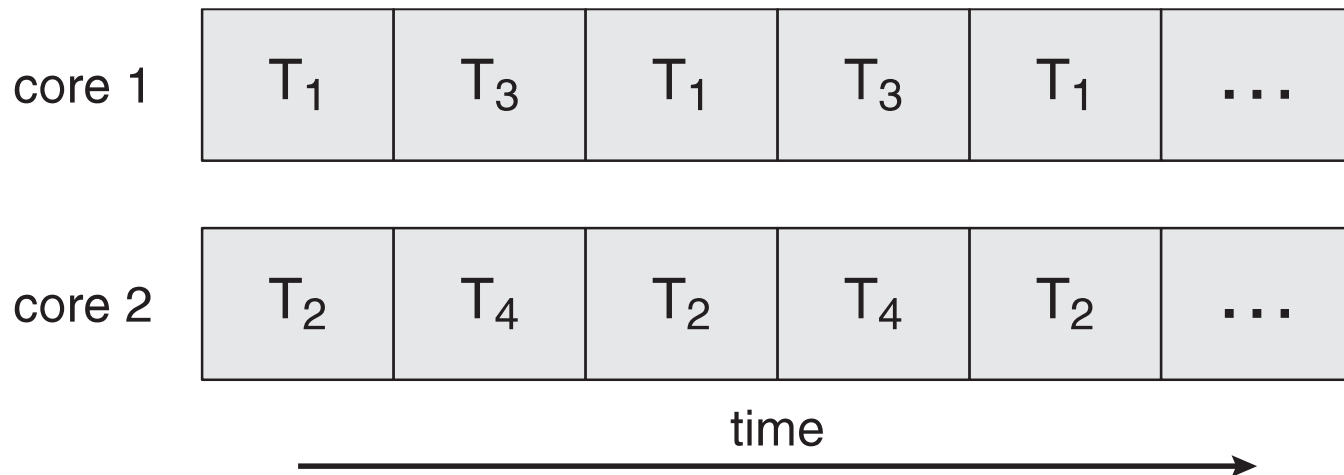
- Bir core üzerinde çalışan **4 thread'e sahip** bir uygulama için **eşzamanlı çalışma**, **thread'lerin belirli aralıklarla çalıştırılmasını ifade eder.**
- Çok core'a sahip sistemlerde **eşzamanlı çalışma**, **her core'a bir thread atanarak thread'lerin paralel çalışmasını ifade eder.**
- **Concurrency (Eşzamanlılık)**, **birden fazla görev arasında kısa aralıklarla geçiş yaparak birlikte ilerletilmesini sağlar.**

Multi-core Programlama (devam...)

Tek çekirdekli sistemde eşzamanlı (Concurrent) yürütme:



Çok çekirdekli bir sistemde paralellik:



Multi-core Programlama (devam...)

- CPU planlayıcıları (schedulers)
 - Prosesler arasında *sürekli switch ederek* devamlılığını sağlarlar ve
 - *Parallelism yapılmıyormuş izlenimi verirler.*
- Aslında bu prosesler
 - **Concurrent** çalışmaktadırlar.
 - **Paralel** çalışmamaktadırlar.

Amdahl Yasası

- Amdahl kuralı: **core sayısına göre bir sistemdeki performans artışı** aşağıdaki gibi ifade edilir:
- S : uygulamada **seri çalışması zorunlu** olan kısmın oranını (**paralel olamayan**), N : ise **core sayısını** ifade eder.

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- Bir uygulamada, %75 paralel ve %25 seri çalışıyorsa ($S=0,25$),
 - 2 core'a ($N=2$) sahip sistemde bu uygulamayı çalıştırınca **1,6 kat hız artar**.
 - Core sayısı 4 olduğunda, **2.28 kat hız artışı** sağlanır.
- **Core sayısı sonsuza giderken** hız artışı ($1/S$) 'e doğru gider

Paralel Çalışma Türleri

- Temel olarak
 - **Data (veri) parallelism** ve
 - **Task (görev) parallelism** olarak

iki farklı paralel çalışma türü vardır.

1. **Data Parallelism:** *Aynı veri kümesine ait* parçaların core'lara dağıtılması ve *aynı tür işlemin eşzamanlı yürütülmesine* odaklanır.
 - **Örneğin:** N elemanlı bir dizi toplamı için **2 core** kullanılacaksa, $[0]..[(N/2)-1]$ eleman 1.core'da, $[N/2]..[N-1]$ eleman 2.core'da toplanır.

Paralel Çalışma Türleri (devam...)

2. **Task Parallelism**: Core'lara görevlerin (thread'ler) dağıtılmasına odaklanır.

– **Her thread ayrı bir işlemi gerçekleştirir.** Farklı thread'ler aynı veride veya farklı veride çalışabilir.

– **Örneğin**: Aynı dizi elemanları üzerinde **farklı** istatistiksel hesaplamalar yapan thread'ler aynı veriyi kullanır farklı core'larda çalışır.

- Çoğu pratik uygulamada, tasarlanan uygulamalar *hem data hem de task paralel çalışma türlerini* **hibrit** olarak kullanır.

Thread Kütüphaneleri

- Thread kütüphanesi, programcıya **thread oluşturmak** ve **yönetmek** için **API sağlar**.
- Thread kütüphanesi oluşturulurken **iki farklı yaklaşım** kullanılır:
 - Tüm thread kütüphanesi **kullanıcı alanında** oluşturulur.
 - İşletim sisteminin doğrudan desteklediği **kernel seviyesinde** kütüphane oluşturulur.

Thread Kütüphaneleri (devam...)

- Çoklu thread oluşturmak için **iki farklı strateji** kullanılmaktadır:
 - **Asenkron threading:** Parent, yeni bir child thread oluşturduğunda **eşzamanlı** olarak çalışmasını sürdürür. Web server mimarisi...
 - **Senkron threading:** Parent, child thread oluşturduğunda çalışmasını durdurur ve tüm child threadler sonlandığında çalışmasına devam eder (fork-join strategy).
- **Asenkron threading:**
 - Threadler arasında **veri paylaşımı az** olduğunda,
- **Senkron threading:**
 - Threadler arasında **veri paylaşımı çok** olduğunda kullanılır.

Thread Kütüphaneleri (devam...)

- Günümüzde **3 temel grup thread kütüphanesi** kullanılmaktadır:
 1. **POSIX Pthreads**
 2. **Windows threads**
 3. **Programlama dillerindeki yüksek seviyeli thread kütüphaneleri**
- **Not: POSIX** (Portable Operating System Interface for Unix), UNIX için "taşınabilir işletim sistemi arabirimi". Bir işletim sistemi hizmeti kümesi tanımlayan IEEE standardıdır. POSIX standardını kullanan programlar bir sistemden diğerine kolayca taşınabilir. POSIX, UNIX sistem hizmetlerine dayanır, ancak başka işletim sistemleri tarafından da geliştirilebilecek biçimde oluşturulmuştur. Kaynak: Vikipedi

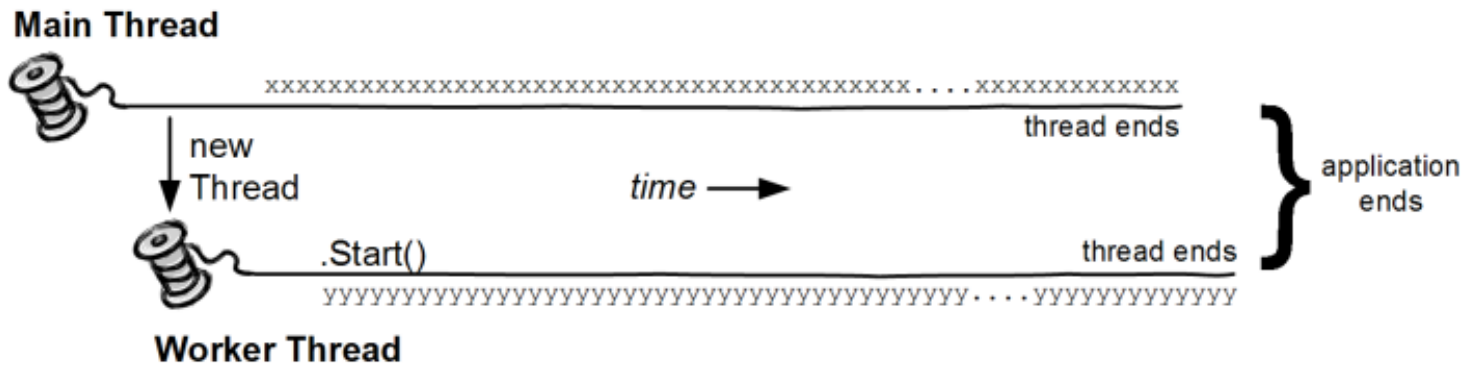
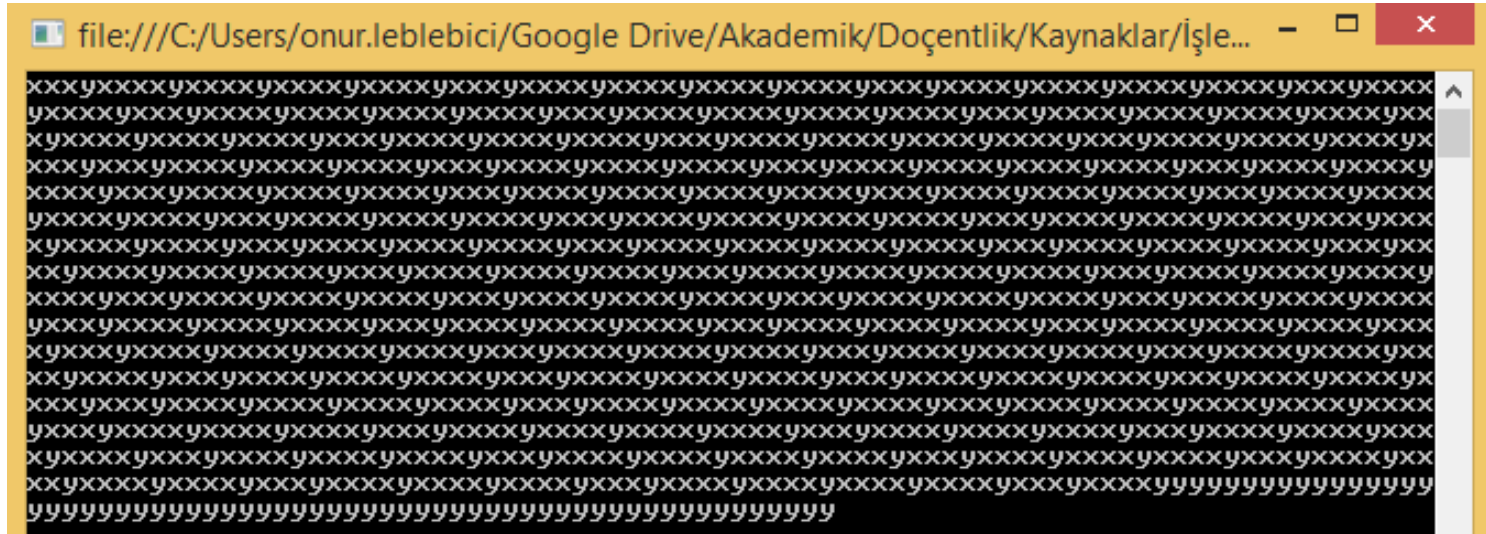
.NET Threads Örnek 1

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        Thread t = new Thread(WriteY);           // Kick off a new thread
        t.Start();                                // running WriteY()

        // Simultaneously, do something on the main thread.
        for (int i = 0; i < 1000; i++) {
            Thread.Sleep(10);
            Console.Write("x");
        }
    }

    1 reference
    static void WriteY()
    {
        for (int i = 0; i < 1000; i++)
        {
            Thread.Sleep(40);
            Console.Write("y");
        }
    }
}
```

.NET Threads Örnek 1



Pthreads

- Pthreads, **IEEE1003.1c** standardıyla thread oluşturma ve yönetmek için tanımlanan API'dir.
- Linux, Unix, Mac OS X ve Solaris işletim sistemleri *Pthreads standardını kullanır.*
- **Windows** Pthreads standardını **desteklemez.**
- Pthreads standardında thread'lerin **hepsi ayrı fonksiyonlar** halinde oluşturulur.
- Tüm thread'ler **global scope'ta** tanımlanan **verileri paylaşırlar.**

Pthreads Örnek 1

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```

Pthreads Örnek 1 (devam...)

```
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid,&attr,runner,argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid,NULL);

    printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

Figure 4.9 Multithreaded C program using the Pthreads API.

Pthreads Örnek 1 Detaylı Açıklama

Pthreads programları için kullanılan header file

Yeni thread için ID tanımlar.

Yeni thread için özellikleri (stack size, ...) belirler.

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```

Varsayılan özellikler (senkron thread, sistem stack addr, ...)

Yeni thread başlatıldı.

Yeni thread için başlama noktası.

Komut satırında girilen parametre

```
/* get the default attributes */
pthread_attr_t attr;
/* create the thread */
pthread_create(&tid, &attr, runner, argv[1]);
/* wait for the thread to exit */
pthread_join(tid, NULL);
```

Komut satırında girilen parametre

```
printf("sum = %d\n", sum);
```

```
/* The thread will begin control in this function */
void *runner(void *param)
```

fork-join stratejisi

```
{
    int i, upper = atoi(param);
    sum = 0;
```

```
    for (i = 1; i <= upper; i++)
        sum += i;
```

```
    pthread_exit(0);
}
```

Linux – Windows Farklılıklar

Durum	POSIX (Linux/macOS)	Windows (WinAPI)
Ana thread return veya exit() ile biterse	Tüm thread'ler zorla kapatılır.	Diğer thread'ler çalışmaya devam eder.
Ana thread pthread_exit() çağırırsa	Diğer thread'ler çalışmaya devam eder.	Geçerli değil (pthread_exit() yoktur).
İşlem tamamen bitirilirse	Tüm thread'ler kapanır.	ExitProcess() çağrılırsa tüm thread'ler kapanır.

Eğer Windows'ta POSIX uyumlu thread'ler kullanmak isterseniz, **Pthreads for Windows** (pthread-win32) kütüphanesini kullanabilirsiniz.

Pthreads Örnek 2

- **Önceki örnekte bir thread oluşturulmuştur.** Çok sayıda thread aşağıdaki örnekteki gibi oluşturulabilir.

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

Oluşturulacak thread sayısı

10 thread tanımlandı.

10 thread için fork-join yapıldı.

Pthreads Örnek 1 (devam...)

- **Derleyelim**
 - **gcc -pthread -o SumThread SumThread.c**
- **Çalıştıralım**
 - **./SumThread 1000**



```
os@debian:~/Desktop/labs$ ./SumThread 1000  
sum = 500500
```

Pthreads Örnek 1 (devam...)

- Fonksiyonu aşağıdaki gibi değiştirelim. **sleep()** ekleyelim.

```
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;
    sleep(40);
    pthread_exit(0);
}
```

- `gcc -pthread -o SumThread SumThread.c`
- `./SumThread 1000`
- İkinci terminal tab açalım ve **pstree -p** çalıştıralım.

Pthreads Örnek 1 (devam...)

```
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;
    sleep(40);
    pthread_exit(0);
}
```

- Linux da thread PID alır. Proses ile benzer davranılır.

```
bash(1972)---SumThread(3651)---{SumThread}(3652)
bash(2718)---pstree(3654)
```

Pthreads Örnek 1 (devam...)

- Aşağıdaki satırı kapatsak ne olur?
pthread_join(tid,NULL);
- Az önceki değişikliği geri alalım sleep() kaldıralım.
- **pthread_join** satırını kapatalım.
- `gcc -pthread -o SumThread SumThread.c`
- `./SumThread 1000`

TOPLAM SONUCU DEĞİŞTİ Mİ?

- Sonrasında thread fonksiyonunda sum değerini ekrana bastıralım. Ekrana ne basar?
- Sonra ana thread de pthread_exit(NULL); ekleyelim en son satıra. Ekran ne basar?

Pthreads Örnek 2

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS      5

void *PrintHello(void *threadid)
{
    long tid;
    tid = (long)threadid;
    printf("Hello World! It's me, thread #%ld!\n", tid);
    pthread_exit(NULL);
}

int main (int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    for(t=0; t<NUM_THREADS; t++){
        printf("In main: creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc){
            printf("ERROR; return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
    }
    pthread_exit(NULL);
}
```

Pthreads Örnek 2 (devam...)

- **Derleyelim ve Çalıştıralım**
- **gcc -pthread -o TList TList.c**
- **./TList**

```
os@debian:~/Desktop/labs$ ./TList
In main: creating thread 0
In main: creating thread 1
In main: creating thread 2
In main: creating thread 3
In main: creating thread 4
Hello World! It's me, thread #3!
Hello World! It's me, thread #4!
Hello World! It's me, thread #2!
Hello World! It's me, thread #1!
Hello World! It's me, thread #0!
```

Pthreads Örnek 2 (devam...)

- **Değiştirilim**

```
int main (int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int rc;
    long t;
    for(t=0; t<NUM_THREADS; t++){
        printf("In main: creating thread %ld\n", t);
        rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
        if (rc){
            printf("ERROR; return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
        //Ekle test et
        pthread_join(threads[t], NULL);
    }
    pthread_exit(NULL);
}
```

- **Derleyelim ve Çalıştıralım**
- **gcc -pthread -o TList TList.c**
- **./TList**

Pthreads Örnek 2 (devam...)

- **Sırayla join oldular ve sırayla işlerini bitirdiler.**

```
os@debian:~/Desktop/labs$ ./TList
In main: creating thread 0
Hello World! It's me, thread #0!
In main: creating thread 1
Hello World! It's me, thread #1!
In main: creating thread 2
Hello World! It's me, thread #2!
In main: creating thread 3
Hello World! It's me, thread #3!
In main: creating thread 4
Hello World! It's me, thread #4!
```

Yararlanılan Kaynaklar

- **Ders Kitabı:**
 - **Operating System Concepts**, Ninth Edition, Abraham Silberschatz, Peter Bear Galvin, Greg Gagne
- **Yardımcı Okumalar:**
 - İşletim Sistemleri, Ali Saatçi
 - Şirin Karadeniz, Ders Notları
 - İbrahim Türkoğlu, Ders Notları
- **M. Ali Akcayol, Gazi Üniversitesi Bilgisayar Mühendisliği Bölümü**
- <http://www.albahari.com/threading/>